

BIST-EMS

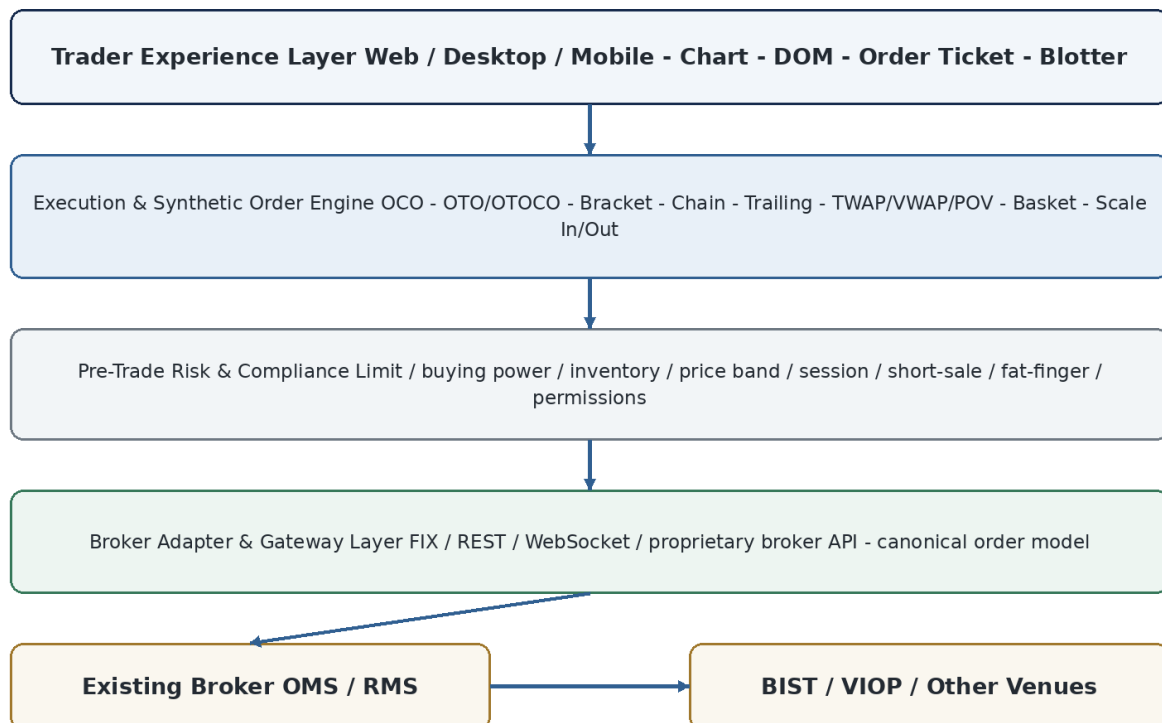
Advanced Trading & Execution Platform

Product Vision, Competitive Benchmark, Order Orchestration, Broker Integration and Technical Architecture

Working-name note

"BIST-EMS" is used only as a working product name in this document. Commercial use of the BIST name/mark should be reviewed with legal and trademark counsel before launch.

BIST-EMS Reference Architecture



Prepared for product discovery and architecture discussion
05 September 2026

Contents

- 1. Executive Summary
- 2. Problem Definition and Product Thesis
- 3. Native Exchange Orders vs Synthetic EMS Orders
- 4. Turkey Market Benchmark
- 5. International Benchmark and Design Lessons
- 6. Target Product Capability Map
- 7. Advanced Order Catalog
- 8. Order Orchestration Engine
- 9. Broker Adapter and Integration Architecture
- 10. Real-Time Data and Event Architecture
- 11. Trader Experience and Screen Architecture
- 12. Risk, Compliance, Audit and Controls
- 13. Reliability, HA, Recovery and Reconciliation
- 14. Canonical Data Model and API Surface
- 15. Deployment Architecture and Technology Choices
- 16. MVP and Phased Roadmap
- 17. Testing and Certification Strategy
- 18. Operational KPIs and Product Metrics
- 19. Key Risks and Mitigations
- 20. Recommended Delivery Model
- 21. Sources and Reference Material

1. Executive Summary

BIST-EMS is a proposed broker-neutral Execution Management System (EMS) and advanced order orchestration layer for Turkish capital markets. The central product idea is deliberately conservative at the infrastructure boundary and aggressive at the user-experience boundary: do not replace the broker's existing OMS/RMS or Borsa Istanbul connectivity; instead, add a modern execution layer in front of it.

Core architectural principle

Preserve the existing order-routing chain. The product sits between the trader UI and the broker OMS/API, converts high-level trading intentions into controlled native child orders, and maintains the synthetic order state server-side.

The opportunity exists because many local trading experiences remain fragmented across charting, market depth, order entry, portfolio monitoring, special-order screens and algorithmic modules. Osmanli Aktiv Trader, MatriksIQ and iDeal demonstrate that Turkish investors already value advanced orders and automation, while ATP/Tradesoft demonstrates that institutional broker infrastructure is mature and deeply embedded. BIST-EMS should therefore compete as a white-label front-office execution layer rather than as another broker core.

The most important differentiator is not a longer list of order buttons. It is a resilient order orchestration engine that can safely manage partial fills, linked exits, protective stops, time/volume slicing, cancellations, retries, reconciliation and broker-specific protocol differences without depending on the browser remaining open.

Dimension	Target position
Product category	White-label EMS / advanced trading terminal / synthetic order engine
Primary buyer	Brokerage firms, banks with brokerage channels, professional trading desks
Primary users	Active retail traders, high-frequency discretionary users, dealers, relationship desks, professional clients
Integration posture	Non-invasive: existing OMS/RMS/BIST connectivity remains system of record for exchange-native orders
Differentiation	Modern trader UX + server-side synthetic orders + broker-neutral adapter model + risk/audit controls
Initial markets	BIST Pay Market and VIOP; architecture extensible to warrants, options and other venues
Commercial model	White-label license + institution adapter/integration + support/SLA + premium algorithm modules

- MVP priority: fast order entry, DOM/ladder, order blotter, cancel/replace, OCO, bracket, chain, trailing stop, conditional orders, basket and server-side persistence.
- Phase 2 priority: TWAP, VWAP, POV, iceberg/slicing, scale-in/scale-out, pair/basket strategies, order templates and advanced risk rules.
- Phase 3 priority: multi-broker/multi-account orchestration, smart routing where legally and technically applicable, institutional analytics, TCA and programmable execution.

2. Problem Definition and Product Thesis

2.1 Customer problem

The customer complaint can be translated into four product problems: order-entry screens are operational rather than trader-centric; advanced order logic is scattered into separate menus; interaction cost is too high for active trading; and capabilities vary significantly by institution because every broker exposes different APIs, OMS features and special-order implementations.

A trader should not need to mentally translate a strategy into several disconnected tickets. The product should allow the trader to express intent - for example, "buy 10,000 lots, take half profit at +3%, the rest at +5%, move the stop to break-even after TP1, and never expose more sell quantity than the actual filled position" - and the EMS should safely orchestrate the required child orders.

2.2 Product thesis

The product should separate trading intent from venue execution. The UI and synthetic-order engine work with a canonical model; the adapter layer translates that model to each institution. This prevents advanced-order logic from being reimplemented for every broker and creates a reusable product core.

What BIST-EMS should not become

It should not be a charting terminal with a few macros added on top, and it should not attempt to replace a broker OMS in the first release. Replacing settlement, accounting, customer master, collateral and complete regulatory back-office workflows would multiply scope and risk without improving the core execution proposition.

3. Native Exchange Orders vs Synthetic EMS Orders

Borsa Istanbul Pay Market supports exchange-native order types and conditions such as limit, market, market-to-limit, imbalance, partially displayed/iceberg, quotation, midpoint, weighted-average-price and short-sale orders, subject to instrument/session rules. These are managed by the exchange matching system once accepted.

An EMS synthetic order is different. It is a parent instruction managed outside the exchange. The EMS observes market data and execution events, then submits, modifies or cancels native child orders according to a state machine. This is the same architectural pattern used globally by professional execution platforms: the synthetic parent is managed by an algo/order server while child orders are sent to the venue.

Layer	Examples	Who owns the state?	Failure implication
Exchange-native	Limit, market, market-to-limit, exchange-supported iceberg/conditions	BIST/BISTECH after acceptance	Exchange continues to manage accepted order even if client UI disconnects
Broker-native special order	Broker-specific conditional, chain or stop implementation	Broker special-order service / OMS	Behavior depends on broker implementation and SLA
BIST-EMS synthetic	OCO, OTOCO, bracket, trailing, TWAP/VWAP/POV, basket, scale-out	BIST-EMS orchestrator	Must survive UI disconnects and reconcile continuously with broker state

This distinction drives the entire architecture: synthetic logic must not live only in the browser. Browser-side logic is acceptable for previews and validation, but the authoritative state machine must run in a server process with durable state and idempotent command handling.

Synthetic Order Tree - Partial Fill Aware Bracket Example

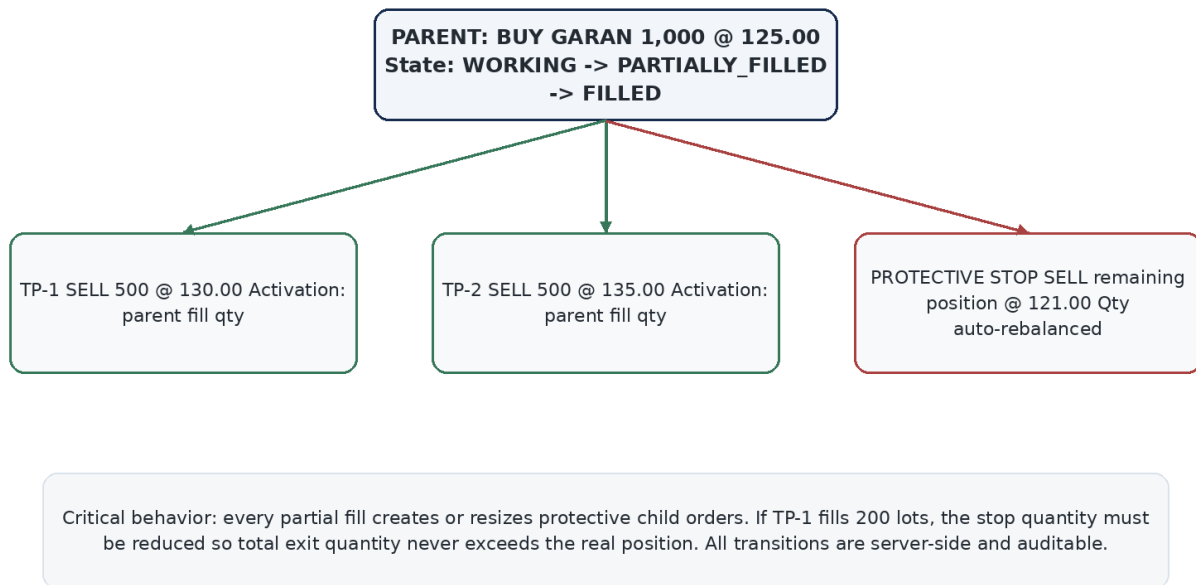


Figure 1 - Example of a server-side bracket order tree with partial-fill-aware child quantities.

4. Turkey Market Benchmark

4.1 Osmanli Yatirim - Aktif Trader

Osmanli Aktif Trader is one of the most relevant local benchmarks because it combines portfolio, analytics, order follow-up, special orders and a Kokpit trading screen. Public product material lists take-profit/stop-loss, trailing stop, timed orders, sliced orders, conditional orders, one-cancels-other, chain orders, conditional cancellation, chain-linked cancellation and futures strategy orders. The Kokpit experience includes one-click order entry, automatic quantity, drag-and-drop price updates, cancel/modify from the same screen and market-depth interaction when the user has a depth package.

The product lesson is that Turkish users already understand and use synthetic order concepts. BIST-EMS should not merely reproduce these features; it should make them composable, more visual, safer under partial fills, available through a consistent builder, and portable across institutions.

4.2 MatriksIQ

MatriksIQ is a stronger benchmark for algorithmic extensibility. Its public documentation emphasizes multi-symbol and multi-timeframe algorithms, C# development, backtesting/optimization and order functions that can be used to build TWAP, basket, staged orders, take-profit, bulk orders, strategy orders and pair orders. This demonstrates demand for programmable execution beyond basic special orders.

4.3 iDeal Data

iDeal provides broker-account and portfolio integration, algorithmic trading infrastructure, fast order windows, strategy examples and automatic take-profit/stop functionality. The strategic lesson is not that its feature set is weak; rather, BIST-EMS should differentiate by making execution workflows first-class, visual and institution-grade instead of treating execution as one module inside a broader data terminal.

4.4 ATP / Tradesoft

ATP Tradesoft is a key benchmark for the institutional side. GTP for Brokerage is positioned as an end-to-end brokerage platform spanning operations, order and risk management, customer and account workflows. Ata Holding

publicly describes Tradesoft as managing a large share of the systems handling Borsa Istanbul transaction volume. For BIST-EMS this is not a UI competitor so much as a likely integration environment and architectural constraint: the product must coexist with entrenched OMS/back-office platforms.

Benchmark	Strong areas to learn from	Opportunity for BIST-EMS
Osmanli Aktiv Trader	Special orders, Kokpit, one-click/drag-drop trading	Unified visual strategy builder, partial-fill-safe orchestration, white-label portability
MatriksIQ	Algo construction, TWAP, baskets, pair/staged orders, backtest	Simpler no-code execution templates plus institutional controls
iDeal	Multi-broker account access, fast tickets, algo/strategy tooling	Execution-first UX, deeper synthetic order lifecycle and state visibility
ATP / Tradesoft	Broker core, order/risk/operations, institutional deployment	Adapter into existing core rather than attempting immediate replacement

5. International Benchmark and Design Lessons

Trading Technologies (TT) is particularly instructive because it explicitly distinguishes synthetic TT Order Types from exchange-native orders. TT documents synthetic parent orders that manage native child orders and supports patterns such as Bracket, OCO, Iceberg, If-Touched, Retry, time-sliced and volume-based orders. Its bracket behavior also addresses partial fills: child OCO quantities are created or adjusted in response to the actual filled quantity.

Interactive Brokers provides another reference for breadth: OCA groups, trailing orders and algorithmic strategies including TWAP and best-efforts VWAP. The lesson is that professional traders expect execution logic to be treated as a product family, not a single "advanced order" menu.

TradingView Broker API is a useful UI/integration reference. Its trading library expects the broker implementation to store and return order state and implements place, modify and cancel operations through a broker adapter. This reinforces the separation between the trading UI and the authoritative broker/EMS backend.

Design principle derived from global platforms

Use a canonical order object and a synthetic parent/child lifecycle. Make strategy behavior explicit, visible and auditable. Avoid hidden client-side magic.

6. Target Product Capability Map

Module	Capabilities	Target
Trading Workspace	Watchlists; chart trading; order ticket; positions; P&L; alerts; multi-layout workspaces	MVP
DOM / Ladder	Depth view; one-click buy/sell; drag-to-modify; cancel-all; flatten; preset quantity	MVP
Order Blotter	Working/filled/cancelled/rejected; parent-child tree; execution details; quick modify	MVP
Synthetic Order Builder	OCO; OTO; OTOCO; bracket; chain; condition; trailing; multi-target exits	MVP
Execution Algos	TWAP; VWAP; POV; time-slice; volume-slice; iceberg; participation caps	Phase 2
Portfolio Actions	Basket; rebalance; multi-symbol order; percentage-based sizing; scale-in/out	Phase 2
Risk & Controls	Buying power; inventory; max order value; daily loss; symbol limits; fat-finger; permissions	MVP
Institution Admin	User/role; desk; account entitlements; feature flags; limits; audit	MVP
Broker Integration	FIX/REST/WebSocket/proprietary adapters;	MVP

	capability negotiation; reconciliation	
Analytics	Slippage; implementation shortfall; fill ratio; reject reasons; execution quality	Phase 2/3
Programmability	Templates; rule builder; SDK/API; optional strategy sandbox	Phase 3

7. Advanced Order Catalog

The order catalog should be modeled as composable execution primitives. Some order names overlap across vendors; the product should define exact semantics, trigger source, partial-fill behavior, child quantity policy, time-in-force and cancellation rules so the same template behaves consistently across institutions.

Order / Algo	Required semantics	Typical use
OCO	Two or more exits are linked; fill in one reduces/cancels the others	Protective exits; competing entry levels
OTO	A child order activates after a parent reaches a defined state	Entry then exit/hedge
OTOCO / Bracket	Entry activates profit target + protective stop as OCO	Standard protected trade
Chain	N-level directed dependency graph between orders	Sequential strategy execution
Trailing Stop	Stop trigger follows favorable price movement by ticks/%/ATR	Dynamic protection
Break-even Stop	After threshold/TP1, stop moves to entry or entry+offset	Locking risk after initial profit
Multi-target Exit	Several TP levels with quantities/percentages and one coordinated stop	Scale-out management
Scale-in	Add quantity at price/condition levels under a max exposure rule	Planned accumulation
TWAP	Slice quantity over time with schedule/randomization constraints	Reduce timing impact
VWAP	Target market volume-weighted benchmark using historical/live volume profile	Benchmark execution
POV	Trade as a bounded percentage of observed market volume	Participation control
Iceberg / Slice	Expose only part of total quantity; replenish children	Reduce displayed size
Basket	Submit coordinated multi-symbol orders with group progress/risk	Portfolio action
Pair / Spread	Coordinate two legs using ratio, trigger and legging policy	Relative-value execution
Conditional	Trigger from price, bid/ask, LTP, percentage, time, position or external rule	General automation
Timed / GTD	Activate/expire at a specific time/date, with optional session restart	Scheduled execution
Cancel-on-condition	Cancel working orders when a condition or linked event occurs	Risk cleanup
Retry / Reprice	Retry rejected/transient orders or reprice under bounded policy	Operational resilience

Every advanced order definition must include a broker capability fallback policy. Example: if the broker supports native iceberg, BIST-EMS may route the native type; otherwise the synthetic engine can slice visible child quantity itself, subject to latency, market rules and institutional approval. The UI must reveal whether an instruction is exchange-native, broker-native or EMS-synthetic.

8. Order Orchestration Engine

8.1 State machine

The orchestration engine is the core intellectual property. Each synthetic parent order is a durable workflow whose state is derived from commands and broker/exchange events. A minimal lifecycle can include CREATED, VALIDATED, ARMED, WORKING, PARTIALLY_FILLED, FILLED, CANCEL_PENDING, CANCELLED, REJECTED, EXPIRED, SUSPENDED and ERROR/RECOVERY states. Child orders have their own lifecycle, and parent state is computed from the strategy policy plus child states.

8.2 Partial-fill correctness

Partial fills are the most common source of dangerous bugs in advanced order systems. Protection quantity must be based on real filled position, not requested parent quantity. If a 1,000-lot entry fills only 300 lots, a protective 1,000-lot sell may create an unintended short position if permitted. Therefore the orchestrator must continuously maintain invariant checks such as total active exit quantity $\leq$ protectable position + explicitly approved short capacity.

- For each fill event, atomically update execution quantity, position projection, child target quantities and risk reservations.
- When one OCO leg partially fills, reduce sibling quantities according to policy before or concurrently with subsequent child action.
- Use monotonic sequence numbers/event versions to ignore duplicates and detect gaps.
- All external commands must be idempotent. A client retry must not duplicate the order.
- Every automatic action must record reason, triggering event, rule version and resulting broker command.

8.3 Strategy definition model

Strategies should be stored as versioned definitions rather than hard-coded UI behavior. A definition contains trigger conditions, actions, dependencies, quantity formulas, price formulas, time policy, risk policy and failure policy. This makes templates portable across web/mobile and enables auditability.

Example invariant

If a parent entry has FilledQty=620 and TP1 has already sold 200, the maximum aggregate protective sell quantity should normally be 420 unless the account explicitly authorizes net short exposure. The rule must be recalculated from authoritative executions, not UI assumptions.

9. Broker Adapter and Integration Architecture

Broker-Neutral Canonical Model and Adapter Pattern

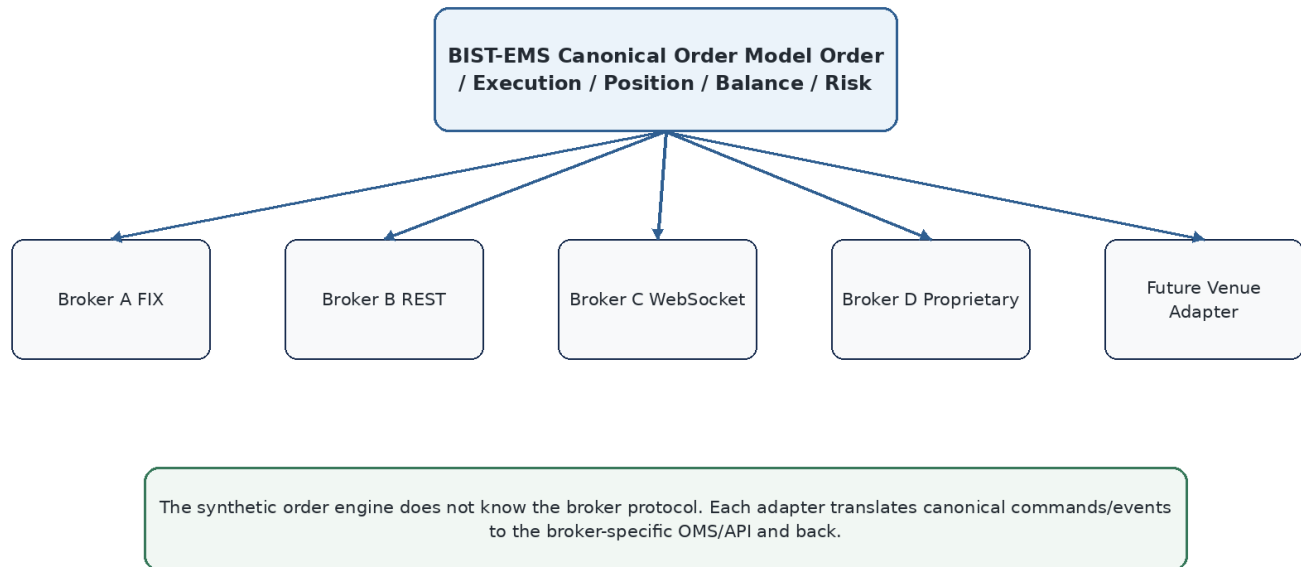


Figure 2 - Canonical model isolates the execution engine from institution-specific protocols.

The adapter boundary is what turns a single custom project into a repeatable product. The EMS core should consume a capability contract such as `placeOrder`, `modifyOrder`, `cancelOrder`, `queryOrders`, `queryExecutions`, `queryPositions`, `queryBalances` and `subscribeToEvents`. Each institution adapter maps these operations to FIX, REST, WebSocket or a proprietary OMS protocol.

Adapter concern	Requirement
Capabilities	Which order types/TIFs/markets/actions the broker supports; native vs synthetic eligibility
Instrument mapping	BIST symbol, broker security ID, market/board/session metadata, tick size and quantity units
Order mapping	Canonical side/type/price/qty/TIF/clientOrderId -> broker fields
Execution mapping	Broker status/fill/reject events -> canonical lifecycle
Error normalization	Map broker-specific codes to stable reject categories and actionable UI messages
Session management	Logon, heartbeat, reconnect, sequence recovery, throttling and maintenance windows
Reconciliation	Periodic/triggered compare of EMS working state vs broker authoritative state
Entitlements	Account, market, feature and user permissions from the institution

For FIX integrations, the system should preserve client order identifiers, broker order identifiers, execution identifiers and sequence metadata. For REST/WebSocket integrations, equivalent correlation and replay semantics must be implemented. The exact protocol is institution-specific; the EMS should not assume all brokers expose the same modification semantics.

10. Real-Time Data and Event Architecture

Advanced execution requires two distinct real-time streams: market data and order/execution events. They must be treated differently. Market data may tolerate controlled sampling for UI rendering, but order/execution events require durable ordering, replay and gap detection.

- Market data path: vendor/feed handler -> normalization -> in-memory distribution -> WebSocket to clients; optional conflation for UI while retaining raw ticks where licensed/required.
- Trading path: API command -> validation -> risk -> orchestration -> broker adapter -> broker ack/fill -> durable event stream -> state store -> client update.
- Do not use the browser as a source of truth for order state. The browser subscribes to authoritative state snapshots plus deltas.
- Use separate back-pressure strategies for market data and trading events. Never drop trading events to protect UI throughput.

11. Trader Experience and Screen Architecture

The trader UI should optimize decision-to-order latency and error prevention. The ideal workspace is not a sequence of modal forms; it is a persistent multi-panel desktop with keyboard shortcuts, context-sensitive order entry and immediate visibility of risk and parent-child relationships.

Screen	Key behaviors
Main Workspace	Watchlist + chart + DOM + order ticket + positions + orders in one layout; detachable/resizable panels
DOM / Ladder	Click price to place; drag working order to modify; cancel at level; cancel-all; quantity presets; position/P&L overlay
Chart Trading	Click/drag entry, stop and targets directly on chart; parent-child lines; preview risk/reward
Advanced Order Builder	Visual nodes or structured form for triggers, child orders, quantity allocation, time rules and failure policy
Order Blotter	Flat and tree views; filter by account/symbol/strategy; execution drill-down; reject reason; one-click cancel/replace
Position Panel	Realized/unrealized P&L; average price; protected/unprotected quantity; quick add stop/TP; flatten/reduce
Basket Desk	Import/paste symbols and targets; calculate quantities; pre-trade validation; group progress and residuals
Admin/Risk Console	Feature entitlements, account limits, kill switch, user/session view, audit search

UX differentiator

A trader should be able to attach or modify protection on an existing position, not only at the moment of entry. Existing positions should show protected quantity, stop coverage and target coverage explicitly.

12. Risk, Compliance, Audit and Controls

Because this system can generate orders automatically, risk controls are a product requirement rather than a later security add-on. The EMS should execute broker/institution rules before an order reaches the adapter while recognizing that the broker OMS/RMS remains the final authoritative gate.

Control	Implementation intent
Buying power / collateral	Reject or resize instructions that exceed usable capacity
Inventory / short exposure	Prevent unintended sells beyond position unless short-selling permission/rules allow
Max order value / quantity	Institution, account, user, symbol and product limits; align with BIST parameters where applicable
Price collars / fat finger	Max deviation from reference price, tick validation, upper/lower price bounds
Daily loss / exposure	Optional desk/account kill thresholds and new-order suspension
Rate limits	Prevent runaway algos and protect broker gateways
Session/market state	Block invalid orders outside allowed phases or handle queued

	activation explicitly
Duplicate protection	Idempotency keys and duplicate-order detection
Kill switch	User/account/desk/system emergency cancel + disable new synthetic actions
Audit trail	Immutable record of intent, validations, automatic decisions, broker commands, responses and operator actions

Audit events should include timestamp with synchronized clock source, actor/service identity, account, strategy/version, correlation ID, original command, derived child command, broker response and previous/new state. Sensitive data must be access-controlled, encrypted and retained according to institutional policy and applicable regulation.

13. Reliability, HA, Recovery and Reconciliation

High Availability and Event Processing Model

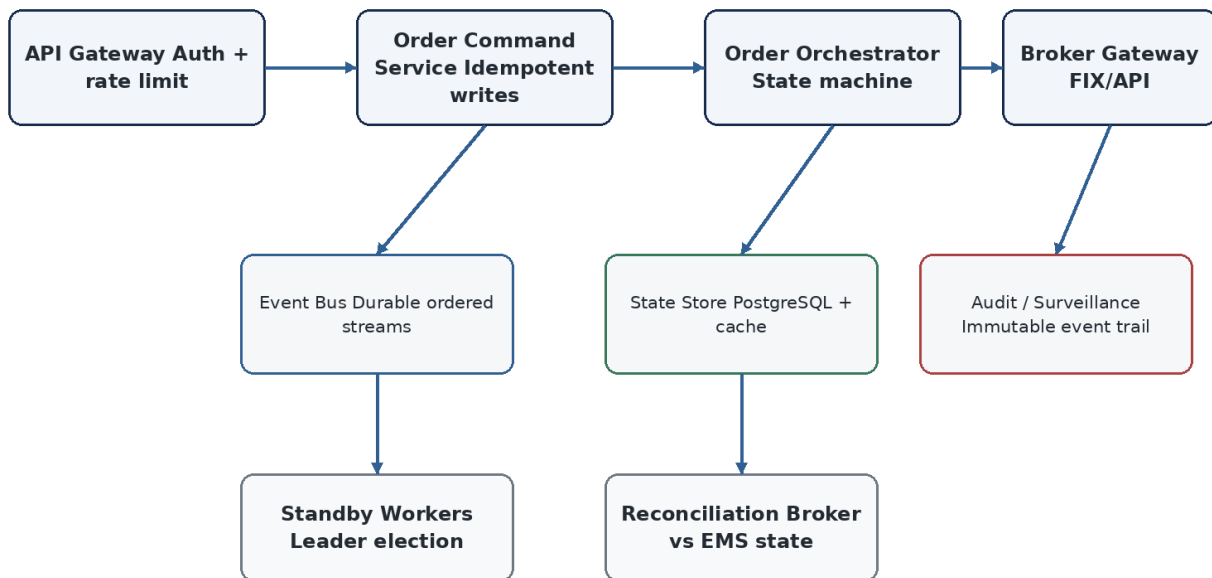


Figure 3 - Trading events require durable processing, recovery and reconciliation.

A synthetic order is a long-lived obligation. Once accepted, it may remain active for hours or days. Therefore the system must continue operating through browser closure, service restarts, network interruptions and individual node failures. High availability is not only an uptime target; it is correctness under failover.

- Persist parent/child state and event history before acknowledging durable acceptance to the client.
- Use leader election or partition ownership so only one active orchestrator instance controls a synthetic parent at a time.
- On restart, rebuild state from snapshot + event log and reconcile all working child orders with the broker.
- Detect unknown broker orders, missing broker orders, quantity mismatches and status conflicts; classify them for automatic repair or operator review.
- Maintain a degradation mode: if market data is stale, suspend trigger-dependent strategies rather than executing on stale prices.
- Separate cancel/flatten emergency path from non-critical analytics services.

14. Canonical Data Model and API Surface

A canonical model should be stable even as individual broker adapters evolve. The minimum entities are Account, Instrument, Order, SyntheticOrder, ChildLink, Execution, Position, Balance, RiskReservation, StrategyDefinition, StrategyInstance, MarketCondition and AuditEvent.

Entity	Critical fields
Order	id, clientId, accountId, instrumentId, side, type, qty, price, stopPrice, tif, status, brokerOrderId, filledQty, avgPrice
SyntheticOrder	id, strategyType, definitionVersion, status, accountId, rootInstrument, parameters, createdBy, riskProfile
ChildLink	parentSyntheticId, childOrderId, role, activationCondition, qtyFormula, cancelGroup, sequence
Execution	executionId, orderId, brokerExecId, qty, price, timestamp, liquidity/fee metadata if available
Position	accountId, instrumentId, qty, avgCost, realizedPnl, unrealizedPnl, protectedQty
AuditEvent	eventId, correlationId, actor, eventType, oldState, newState, reason, payloadHash, timestamp

Externally, REST can be used for commands and queries while WebSocket/SSE supplies state updates. Internally, an event bus can decouple trading state updates from notification, analytics and audit consumers. Protocol choices should prioritize determinism and operational simplicity over fashionable complexity.

15. Deployment Architecture and Technology Choices

A practical implementation can use a service-oriented architecture without immediately becoming an over-fragmented microservice estate. The latency-sensitive trading path should have few network hops and carefully controlled dependencies.

Component	Suggested responsibility / technology direction
Trading Web	React/Next.js or Vite; WebSocket; virtualized order/depth grids; keyboard-first UX
API Gateway	Authentication, session binding, rate limiting, request IDs, routing
Order Service	Command validation, idempotency, canonical order persistence
Orchestrator	Deterministic state-machine workers; partition by account/strategy; no UI dependency
Risk Service	Low-latency pre-trade checks with cached limits/positions plus broker final validation
Broker Gateways	Dedicated connectors per institution/protocol; FIX engine where required
Event Backbone	Kafka/Redpanda/NATS JetStream or equivalent depending operational constraints; strict durability for trading domain
Primary DB	PostgreSQL for durable relational state; append-only audit/event tables or event store pattern
Cache	Redis only for cache/coordination where safe; never sole store of accepted order state
Observability	Metrics + logs + distributed correlation; business metrics for order/fill/reject/reconcile
Secrets	Institution-managed vault/HSM integration where required; key rotation and least privilege

For on-premise broker deployments, containerized services on Kubernetes or a controlled VM/systemd topology can both be supported. The decision should follow the institution's operations standards. The product should package the trading core in a way that is deterministic and supportable in disconnected or tightly restricted networks if required.

16. MVP and Phased Roadmap

Phase	Scope	Gate
Phase 0 - Discovery & protocol proof	Broker API/FIX capability matrix; data licensing; BIST market/session rules; security and deployment constraints; click-through prototype	Exit: one end-to-end sandbox order and state reconciliation

Phase 1 - Trading core	Login/account context; watchlist; depth; fast ticket; positions; blotter; place/modify/cancel; OCO/bracket/chain/trailing; audit; kill switch	Exit: controlled pilot with one broker and selected users
Phase 2 - Algo execution	TWAP, POV, iceberg/slicing, basket, multi-target exits, scale-in/out, templates, richer admin/risk	Exit: production algorithm suite with measurable execution KPIs
Phase 3 - Institutional scale	Second/third broker adapters, multi-account workflows, TCA, programmable APIs, pair/spread workflows, advanced HA	Exit: repeatable white-label product
Phase 4 - Optimization	Smart routing where applicable, ML-assisted execution analytics, adaptive parameter suggestions with human control	Exit: differentiated execution intelligence

The first broker integration should be chosen for technical openness and sponsor commitment, not only brand value. A cooperative institution that provides stable test APIs, reject-code documentation and a realistic certification environment will reduce program risk dramatically.

17. Testing and Certification Strategy

Trading software must be tested as a distributed state machine, not only as a web application. The test plan should simulate race conditions, duplicated messages, out-of-order events, broker disconnects and partial fills.

- Deterministic unit tests for every state transition and quantity/price formula.
- Property/invariant tests: no duplicate child creation, no over-exit beyond allowed exposure, monotonic filled quantity, valid state transitions only.
- Broker adapter contract tests using recorded and synthetic broker messages.
- Market-session tests: opening, continuous trading, closing, halts, stale data and day rollover.
- Chaos/failover tests: orchestrator crash, database failover, event-bus delay, network partition, broker reconnect.
- Replay tests: reconstruct a full trading day from events and verify resulting state equals production snapshot.
- Performance tests: burst order entry, depth updates, cancel storms and broker throttle behavior.
- User acceptance scenarios with professional traders, emphasizing accidental-order prevention and workflow speed.

18. Operational KPIs and Product Metrics

Metric	Why it matters
Order command p50/p95/p99 latency	Measures EMS overhead before broker/network latency
Broker ack latency and reject rate	Detects adapter/broker health and invalid-order patterns
Synthetic action latency	Trigger-to-child-command latency for stops/conditions
Reconciliation mismatch count	Direct signal of state correctness
Duplicate-prevention incidents	Should remain zero; critical correctness KPI
Protected position coverage	Shows whether positions expected to be protected actually have valid protection
Fill ratio / cancel ratio / slippage	Execution quality and strategy behavior
Algo completion vs benchmark	TWAP/VWAP/POV quality assessment
Client action count per trade	UX efficiency: fewer clicks/keystrokes for common workflows
Gateway reconnect/recovery time	Operational resilience
Synthetic order failure rate	Reliability of advanced-order engine

19. Key Risks and Mitigations

Risk	Impact	Mitigation
Broker API inconsistency	Every integration behaves differently	Capability matrix, canonical adapter contract, broker-specific certification suite

Partial-fill race conditions	Over-selling, unprotected exposure	Atomic state updates, ordered events, invariants, reconciliation
Stale market data	Wrong trigger execution	Feed health state, stale thresholds, suspend trigger-based strategies
UI disconnect	Advanced order stops working	Server-side orchestration and durable state
Duplicate client/broker messages	Duplicate orders or incorrect quantities	Idempotency, broker IDs, sequence/version checks
Runaway algorithm	High order rate/exposure	Rate limits, max child count, risk budgets, kill switch
Regulatory/market-rule changes	Invalid behavior after rule update	Versioned rule service, release governance, institutional approval
Vendor/data licensing	Depth/market data cannot be redistributed freely	License review and institution/vendor contracts
Trademark/product naming	"BIST" name may create legal/brand issues	Treat BIST-EMS as working title until legal clearance

20. Recommended Delivery Model

The recommended approach is to build BIST-EMS as a product core plus institution-specific adapters and branding packages. The core should own synthetic order semantics, risk framework, audit model, trader workspace and admin console. Institution packages should contain protocol mapping, entitlements, deployment configuration, branding and institution-specific rules.

1. Select one launch institution and document the current order path end-to-end: client channel -> API/gateway -> OMS/RMS -> BIST/VIOP.
2. Create a broker capability matrix: supported markets, order types, modify/cancel semantics, event streams, rate limits, session rules, test environment and authentication.
3. Prototype the trader workspace and advanced-order builder with real trader input before deep backend development.
4. Implement the canonical order model and one adapter with full reconciliation before adding many synthetic order types.
5. Deliver OCO/bracket/chain/trailing with rigorous partial-fill testing; these establish the orchestration foundation.
6. Add TWAP/basket/scale workflows only after the core state machine and risk controls are production-proven.
7. Generalize the first integration into a documented adapter SDK and onboard the second institution to validate product portability.

Recommended product message to brokers

"We do not ask you to replace your OMS, RMS or exchange connectivity. We add a modern execution and advanced-order layer in front of the infrastructure you already trust."

21. Sources and Reference Material

The following public sources were used to validate the market benchmark and architectural concepts. Product capabilities can change; institution-specific technical details must be confirmed during integration discovery.

Source	URL
Borsa Istanbul - Pay Market / Market Operation	https://www.borsaistanbul.com/piyasalar/pay-piyasasi/piyasa-isleyisi
Borsa Istanbul - Pay Market Procedure (PDF)	https://www.borsaistanbul.com/files/pay-piyasasi-proseduru.pdf
Osmanli Menkul - Aktif Trader Web	https://www.osmanlimenkul.com.tr/hisse-ve-viop/platformlarimiz/osmanli-platformlari/osmanli-aktif-trader/aktif-trader-web
Osmanli Menkul - Special Orders	https://www.osmanlimenkul.com.tr/hisse-ve-viop/platformlarimiz/osmanli-platformlari/ozel-emirler-ve-algoritmik-islemler
Osmanli Menkul - Kokpit	https://www.osmanlimenkul.com.tr/hisse-ve-viop-urunlerimiz/kokpit-ekranlari
MatriksIQ - Data Terminal / Algorithmic Trading	https://www.matriksdata.com/website/urunlerimiz/kullanici-platformlari/matriksiq-veri-terminali
iDeal Data - Product Overview	https://idealddata.com.tr/

ATP Tradesoft - GTP for Brokerage	https://www.atptech.com/atp-tradesoft/tradesoft-gtp-for-brokerage/?lang=en
Ata Holding - Tradesoft overview	https://www.ataholding.com.tr/en/industries/technology/tradesoft
TradingView - Broker API / Manage Orders	https://www.tradingview.com/charting-library-docs/latest/tutorials/tutorials/implement-broker-api/manage-orders/
Trading Technologies - TT Order Types Overview	https://library.tradingtechnologies.com/trade/basic-order-entry/tt-order-types/description-tt-order-types/tt-order-types-overview/
Trading Technologies - TT Bracket Order	https://library.tradingtechnologies.com/trade/basic-order-entry/tt-order-types/description-tt-order-types/tt-bracket-order/
Interactive Brokers - Order Types and Algos	https://portal.interactivebrokers.com/en/trading/ordertypes.php
Borsa Istanbul - TT connectivity announcement for VIOP	https://www.borsaistanbul.com/en/announcement/13365/trading-technologies-launches-connectivity-borsa-istanbul-derivatives-trading-ttr-platform

Disclaimer: This document is a product and technical architecture study, not investment advice, legal advice, regulatory approval or a statement that every listed order type is directly supported by Borsa Istanbul or every brokerage. Exchange-native, broker-native and EMS-synthetic behavior must be explicitly distinguished during implementation.